Intermediate (build real memory) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `conversation_memory_practice.py`: an LLM API call has no memory of its own — every call is stateless. "Memory" is really just resending the whole conversation so far, every single time. Building that resend-everything loop by hand, once, here, is what makes the Build Task's `chat_client.py` obvious instead of magic. **If not:** you'd be handed `send_message(history, user_input)` in the Build Task without ever having seen why it needs the whole `history` list passed in every time, not just the newest message.

Solution

```
# conversation_memory_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def get_reply(history: list) -> str:
    # why: this exact shape — take the whole history, return just the new
    # reply text — is what the Build Task's send_message() needs to look like.
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        # how: the model only ever sees history — it has no memory of its
```

```
        # own, so every earlier turn has to be resent, every single time.
        history.append({"role": "user", "content": user_input})

        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)

if __name__ == "__main__":
    run_chat()
```

Expected output: a running conversation where the assistant remembers earlier turns — type `quit` to stop.